STUDY MATERIAL

Graph Theory & Algorithms

Comprehensive Guide with Proofs, Visualizations and Viva Q&A

Trees, BFS/DFS, Connectivity, Bipartite Graphs

DAGs, Greedy Algorithms, Shortest Paths, MST

April 2026

Table of Contents

1. Graph Fundamentals: Trees

1.1 Tree Edge Property

1.2 Characterization Theorem

1.3 Viva Q&A

2. Graph Representations

2.1 Adjacency Matrix vs Adjacency List

2.2 Viva Q&A

3. Graph Traversals: BFS and DFS

3.1 BFS Layer Properties

3.2 DFS Non-Tree Edge Property

3.3 Queue vs Stack: Why Data Structure Matters

3.4 Viva Q&A

4. Connected Components

4.1 Finding Connected Components

4.2 Identical or Disjoint Property

4.3 Viva Q&A

5. Bipartite Graphs

5.1 Definition and Characterization

5.2 BFS-Based Bipartite Testing

5.3 Viva Q&A

6. Directed Graphs: SCCs and DAGs

6.1 Strongly Connected Components

6.2 DAGs and Topological Ordering

6.3 Viva Q&A

7. Greedy Algorithm Design

7.1 Core Elements and Proof Strategy

7.2 Greedy vs Dynamic Programming

7.3 Viva Q&A

8. Shortest Path Algorithms

8.1 Dijkstra's Algorithm

8.2 When Dijkstra Fails

8.3 Viva Q&A

9. Minimum Spanning Trees

9.1 Cut Property and Cycle Property

9.2 Prim's vs Kruskal's Algorithm

9.3 Viva Q&A

10. Algorithm Comparison Summary

1 Graph Fundamentals: Trees

1.1 Tree Edge Property

Theorem 3.1

Every tree with n nodes has exactly $n - 1$ edges.

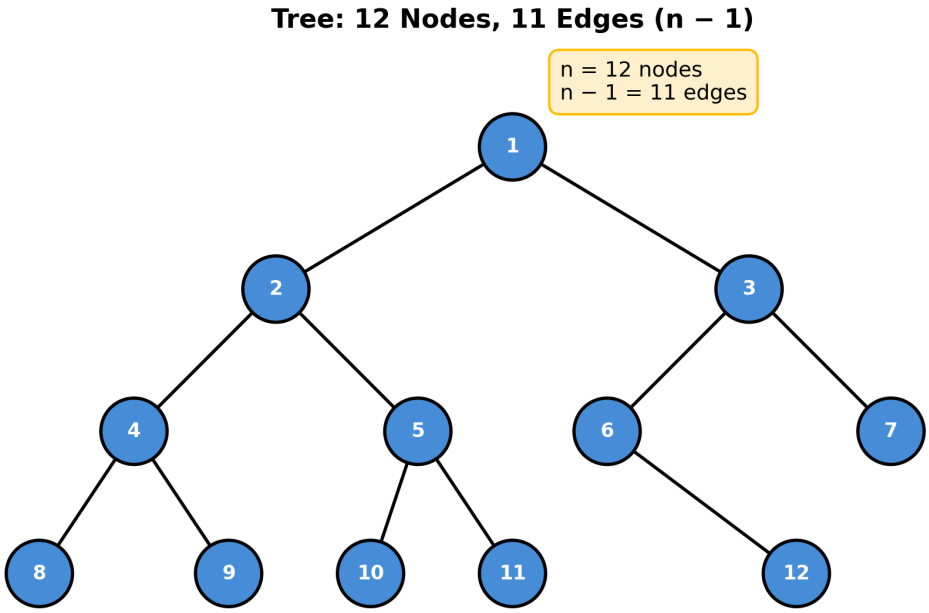

Figure 1: *A tree with 12 nodes and exactly 11 edges ($n - 1$). Removing any edge disconnects the graph; adding any edge creates a cycle.*

Definition

A **tree** is a connected, acyclic undirected graph.

Proof by Induction

Base case: For $n = 1$, a single-node tree has $0 = 1 - 1$ edges.

Inductive step: Assume true for all trees with k nodes (i.e., $k - 1$ edges). Consider a tree T with $k + 1$ nodes.

disconnected.

- Remove a leaf and its incident edge. The remaining graph is a tree with k nodes, so by hypothesis it has $k-1$ edges.
- Add the leaf back: total edges $= (k-1)+1 = k = (k+1)-1$. $\square$

Alternative Proof (Counting): Start with n isolated nodes (0 edges, n components). Each edge addition reduces the number of connected components by exactly 1 (no cycles allowed). To get 1 connected component, need exactly $n-1$ edges.

1.2 Characterization Theorem

Theorem 3.2

Let G be an undirected graph on n nodes. Any two of the following statements implies the third:

1. *G is connected*
2. *G does not contain a cycle*
3. *G has $n-1$ edges*

A graph satisfying all three properties is a **tree**. The power of this theorem lies in proving any two conditions to establish the third.

Proof cases for the Characterization Theorem

Assumed	Proved	Key Reasoning
(i) Connected + (ii) Acyclic	⇒ (iii) $n-1$ edges	By Theorem 3.1, connected acyclic graphs are trees
(i) Connected + (iii) $n-1$ edges	⇒ (ii) Acyclic	A connected graph with a cycle has $\geq n$ edges; with $n-1$ edges, no cycle possible
(ii) Acyclic + (iii) $n-1$ edges	⇒ (i) Connected	If disconnected with $k \geq 2$ components, each acyclic component with n_i nodes has $\leq n_i - 1$ edges, totaling $\leq n-k \leq n-2$ edges — contradiction

1.3 Viva Questions and Answers

Viva Q&A — Trees

Q: Does a connected graph with $n-1$ edges have to be a tree?

A: Yes. Connected + $(n-1)$ edges ⇒ acyclic by Theorem 3.2, hence a tree.

Q: Can an acyclic graph with $n-1$ edges be disconnected?

A: No. Acyclic + $(n-1)$ edges ⇒ connected. If it had $k \geq 2$ components, the maximum edges would be $n-k \leq n-2$, contradicting $n-1$ edges.

Q: Do these properties hold for directed graphs?

A: No. Directed trees (arborescences) have different properties. The characterization is specific to undirected graphs.

Q: What is the minimum number of leaves in a tree?

A: Every tree with $n \geq 2$ nodes has at least 2 leaves. This follows from $\sum \deg(v) = 2(n-1)$.

2 Graph Representations

2.1 Adjacency Matrix vs Adjacency List

Statement 3.10

The adjacency matrix representation requires $\Theta(n^2)$ space, while the adjacency list requires only $\Theta(m+n)$ space.

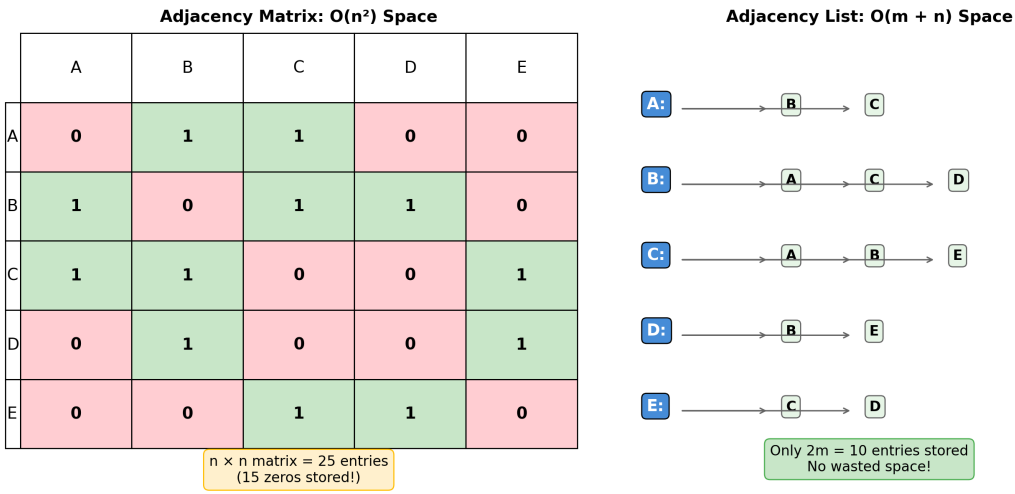

Figure 2: *Adjacency Matrix stores n^2 entries regardless of edge count; Adjacency List stores exactly $m+n$ entries.*

Comparison of graph representations

Aspect	Adjacency Matrix	Adjacency List
Space	$\Theta(n^2)$ — stores all $n \times n$ entries	$\Theta(m+n)$ — stores edges + headers
Edge lookup	$O(1)$ — direct array access $A[u][v]$	$O(\deg(u))$ — scan neighbor list
Iterate neighbors	$O(n)$ — scan entire row	$O(\deg(u))$ — directly access list
Best for	Dense graphs ($m \approx n^2$)	Sparse graphs ($m \ll n^2$)
Weighted graphs	Store weight in cell (or ∞/0)	Store (v, w) pairs

Key Insight: For sparse graphs like social networks or road networks where $m \ll n^2$, adjacency lists are essential. For dense graphs or when fast edge lookup is critical, the

matrix's $O(1)$ access may win.

2.2 Viva Questions and Answers

Viva Q&A — Graph Representations

Q: What is the space for adjacency list of a tree?

A:$O(n)$ since $m = n - 1$ for trees, so $O(m + n) = O(2n - 1) = O(n)$.

Q: Can we do better than $\Theta(n^2)$ for dense graphs?

A: No. For dense graphs, you need $\Theta(n^2)$ bits just to record edge existence.

Q: Does the adjacency matrix work for directed graphs?

A: Yes. $A[i][j] = 1$ if edge $i \to j$ exists. The matrix is not symmetric for directed graphs.

Q: How does space differ for undirected vs directed adjacency lists?

A: Undirected: $2m$ entries (each edge stored twice). Directed: m entries. Matrix is n^2 regardless.

3 Graph Traversals: BFS and DFS

3.1 BFS Layer Properties

Statement 3.3

For each $j \geq 1$, layer L_j produced by BFS consists of all nodes at distance exactly j from s. There is a path from s to t iff t appears in some layer.

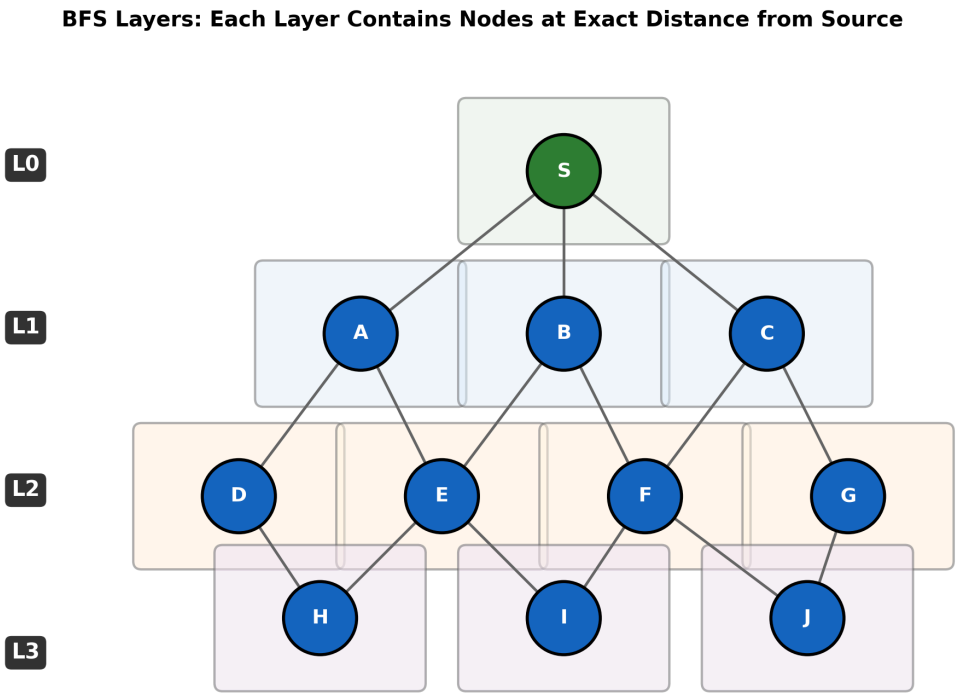

Figure 3: *BFS explores in concentric layers. Layer L_j contains exactly the nodes at shortest-path distance j from source S.*

Why layers correspond to exact distances: BFS uses a queue (FIFO). It processes $L_0 = \{s\}$ first, then all neighbors of L_0 form L_1 (distance 1), then all unvisited neighbors of L_1 form L_2 (distance 2), and so on. Since BFS discovers nodes via shortest paths first, any node at distance exactly j must be adjacent to some node at distance $j - 1$, so it gets discovered in layer L_j.

Proof sketch: Let $d(v)$ be the shortest-path distance from s to v. Since (x, y) is an edge, there exists a path to y via x of length $d(x) + 1 = i + 1$, so $d(y) = j \leq i + 1$. Similarly $i \leq j + 1$. Combining: $|i - j| \leq 1$.

> **Intuition:** BFS layers form "concentric rings" around s. You cannot have an edge that "jumps" across more than one ring because such an edge would provide a shortcut contradicting the shortest-path property.

3.2 DFS Non-Tree Edge Property

> **Statement 3.7**
>
> *Let T be a DFS tree. Let x, y be nodes in T, and let (x, y) be an edge of G not in T. Then one of x or y is an ancestor of the other.*

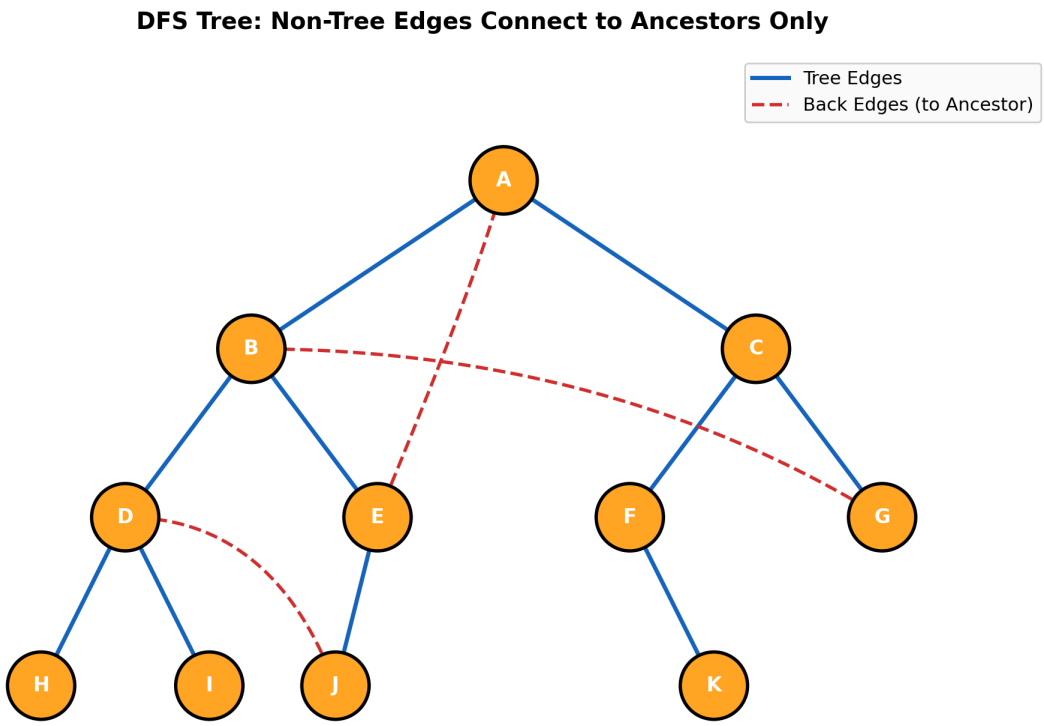

Figure 4: *In a DFS tree, non-tree edges (dashed red) always connect a node to an ancestor. No cross edges exist between unrelated branches in undirected graphs.*

Why: DFS explores as deep as possible along one branch before backtracking. When at node x, it recursively visits unvisited neighbors. If (x, y) is a non-tree edge and y was

Discovery/Finish Times

DFS assigns each node v a discovery time $d[v]$ and finish time $f[v]$. For any edge (x, y): if $d[x] < d[y] < f[y] < f[x]$, then y is a descendant of x.

3.3 Queue vs Stack: Why Data Structure Matters

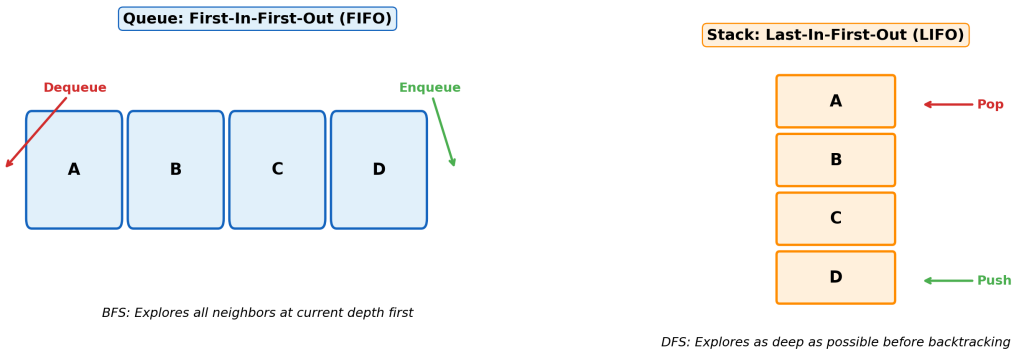

Figure 5: *Queue (FIFO) enables breadth-first exploration; Stack (LIFO) enables depth-first exploration. The data structure determines the traversal semantics.*

BFS vs DFS core differences

Aspect	BFS (Queue)	DFS (Stack/Recursion)
Exploration order	All nodes at distance d, then $d + 1$	As deep as possible, then backtrack
Shortest path	Finds shortest path in unweighted graphs	Does not guarantee shortest path
Use cases	Shortest path, bipartite testing, level order	Cycle detection, topological sort, SCCs
Can simulate other?	Only with extreme contrivance	Only with extreme contrivance

Critical Insight: Technically you can use a queue in DFS-like code, but it becomes BFS. The data structure *determines* the traversal semantics. Two stacks can simulate a queue (amortized $O(1)$ per operation), but this is a queue implementation, not "stack-based BFS."

3.4 Viva Questions and Answers

Viva Q&A — Graph Traversals

Q: Can a non-tree edge connect L_j to L_{j+2} in BFS?

A: No. That would violate Statement 3.4. BFS edges never skip more than one layer.

Q: Does BFS layer property hold for weighted graphs?

A: No. BFS layers don't give shortest paths with weights — use Dijkstra instead.

Q: Can DFS have cross edges in undirected graphs?

A: No. Every non-tree edge connects a node to an ancestor. In directed graphs, cross edges can exist between different branches.

Q: How is BFS used for bipartite testing?

A: Color layers alternately. Any edge connecting nodes in the same layer indicates an odd cycle, so the graph is not bipartite.

Q: Can we use a queue for DFS by pushing in reverse order?

A: No. A queue is FIFO; processing in "reverse order" with a deque is contrived. Queue = BFS; Stack = DFS. The core semantics are determined by the data structure.

4 Connected Components

4.1 Finding Connected Components

The standard algorithm uses BFS or DFS to find all connected components of an undirected graph G:

Algorithm: Finding All Connected Components

```
Initialize all nodes as unvisited
For each unvisited node u:
    Start BFS/DFS from u
    All discovered nodes form one connected component
    Mark all discovered nodes as visited
```

Why this works: Each call to BFS/DFS discovers exactly one connected component (all nodes reachable from the starting node). Components are disjoint by definition, so no node is ever revisited.

Complexity of connected components

Aspect	Complexity	Reason
Time	$O(m+n)$	Each node and edge examined once with adjacency lists
Space	$O(n)$	Visited set + queue/stack
BFS vs DFS	Both work; order of discovery differs but component set is identical	

4.2 Identical or Disjoint Property

Statement 3.8

For any two nodes s and t in a graph, their connected components are either identical or disjoint.

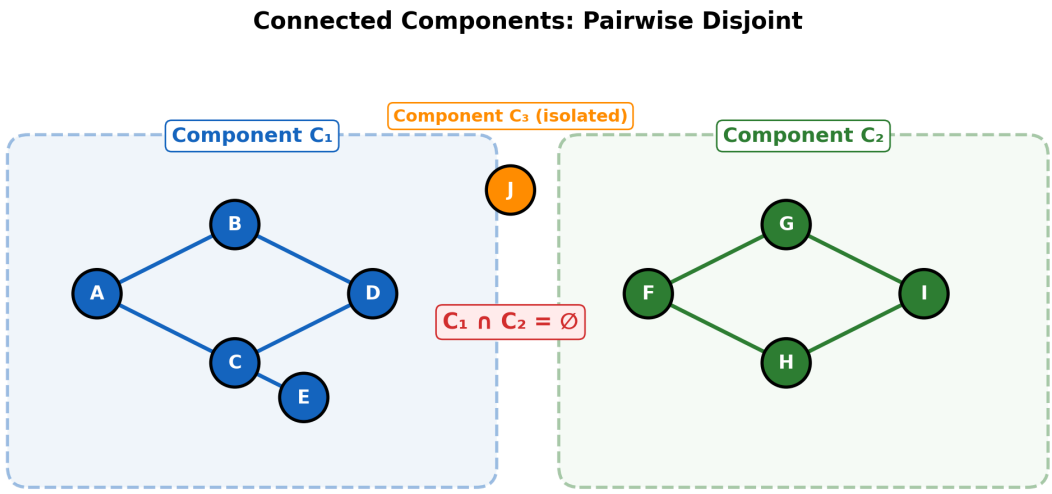

Figure 6: *Connected components partition the vertex set. Any two components are either the same set or have empty intersection.*

Proof by contradiction: Suppose $C(s) \cap C(t) \neq \emptyset$ but $C(s) \neq C(t)$. Let $u \in C(s) \cap C(t)$. Then paths $s \leftrightarrow u$ and $t \leftrightarrow u$ exist. By path concatenation, $s \leftrightarrow t$ exists, so $s \in C(t)$ and $t \in C(s)$. By transitivity, any node in $C(s)$ is in $C(t)$ and vice versa, so $C(s) = C(t)$ — contradiction.

> **Equivalence Relation:** Connectivity defines an equivalence relation: reflexive ($s \leftrightarrow s$), symmetric ($s \leftrightarrow t \Rightarrow t \leftrightarrow s$), and transitive (path concatenation). Connected components are the equivalence classes.

4.3 Viva Questions and Answers

Viva Q&A — Connected Components

Q: Does this algorithm work for directed graphs?

A: No. For directed graphs it finds weakly connected components, not strongly connected components (SCCs). For SCCs, use Kosaraju's or Tarjan's algorithm.

Q: Why not check all pairs of nodes?

A: That would be $O(n^2 \cdot m)$ or worse. BFS/DFS gives $O(m + n)$ linear time.

Q: Maximum number of connected components?

A:n (all isolated nodes). Minimum is 1 (connected graph).

Q: If two components share a node, what happens?

A: They must be the same component. Components are pairwise disjoint.

5 Bipartite Graphs

5.1 Definition and Characterization

Definition

A graph $G = (V, E)$ is **bipartite** if V can be partitioned into two disjoint sets A and B ($A \cup B = V$, $A \cap B = \emptyset$) such that every edge connects a node in A to a node in B. Equivalently: no edge has both endpoints in the same set.

Bipartite Theorem (Statement 3.15)

A graph is bipartite $\Leftrightarrow$ it contains no odd-length cycles.

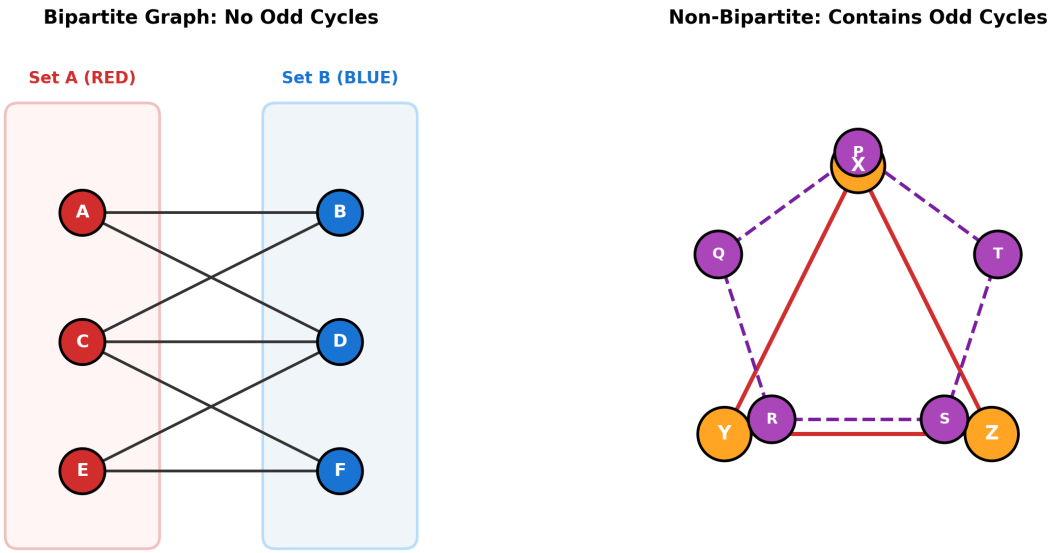

Figure 7: *Left: Bipartite graph with valid 2-coloring. Right: Non-bipartite graphs containing odd cycles (3-cycle and 5-cycle).*

Why no odd cycles ⇒ bipartite: Pick any node s, put all nodes at even distance from s in A, all at odd distance in B. By BFS layer properties, any edge within the same layer would create an odd cycle (via tree paths). Any edge between layers differing by > 1 would create a shortcut. So edges only go between adjacent layers — hence between A and B.

5.2 BFS-Based Bipartite Testing

The BFS characterization leads directly to an efficient algorithm:

Algorithm: BFS Bipartite Testing (2-Coloring)

```
Color start node RED (layer 0)
Run BFS, alternating colors by layer:
    Layer 0, 2, 4, ... → RED (even distance)
    Layer 1, 3, 5, ... → BLUE (odd distance)
If any edge connects two nodes of the same color:
    Return "NOT BIPARTITE" (odd cycle found)
Return "BIPARTITE"
```

Statement 3.15 (Full)

Let G be connected and $L_0, L_1, L_2, \ldots$ be BFS layers from s. Exactly one of:

1. ***No same-layer edge*** *— G is bipartite (even layers = RED, odd = BLUE)*
2. ***Same-layer edge exists*** *— G contains an odd cycle, not bipartite*

Finding the odd cycle: If edge (x, y) exists with $x, y \in L_j$, let z be their lowest common ancestor in the BFS tree at layer L_i where $i < j$. The cycle $z \to x \to y \to z$ has length $2(j - i) + 1$, which is odd.

Complexity of bipartite testing

Aspect	Complexity	Reason
Time	$O(m + n)$	Single BFS/DFS pass
Space	$O(n)$	Color array + queue
Disconnected?	Run on each component; all must be bipartite	

5.3 Viva Questions and Answers

Viva Q&A — Bipartite Graphs

Q: Is every tree bipartite?

A: Yes. Trees have no cycles, so certainly no odd cycles.

Q: Is a complete graph K_n bipartite?

A: Only for $n \leq 2$. K_3 (triangle) has an odd cycle of length 3.

Q: Can DFS be used instead of BFS for bipartite testing?

A: Yes. Same coloring logic works. BFS naturally gives the layer intuition, but DFS also detects same-color conflicts.

Q: What about self-loops?

A: Immediately not bipartite — a self-loop is a cycle of length 1 (odd).

Q: Are hypercubes bipartite?

A: Yes. n-dimensional hypercubes can be partitioned by parity of bit-sum.

6 Directed Graphs: SCCs and DAGs

6.1 Strongly Connected Components

Definition

The **strongly connected component** (SCC) of a node s in a directed graph is the set of all nodes v such that there is a directed path $s \to v$ AND a directed path $v \to s$ (mutual reachability).

Statement 3.17

For any two nodes s and t in a directed graph, their strong components are either identical or disjoint.

Proof: Define $s \sim t$ iff s and t are mutually reachable. This is an equivalence relation: reflexive (trivial path of length 0), symmetric (definition of mutual), and transitive (path concatenation). SCCs are the equivalence classes, which partition V.

Undirected vs Directed Components

Aspect	Undirected (3.8)	Directed (3.17)
Relation	Reachability ($s \leftrightarrow t$)	Mutual reachability ($s \leftrightarrow t$ and $t \leftrightarrow s$)
Component	Connected component	Strongly connected component (SCC)
Partition?	Yes	Yes
Algorithm	BFS/DFS, $O(m+n)$	Kosaraju's or Tarjan's, $O(m+n)$

Condensation Graph: Contract each SCC to a single "supernode." The result is always a DAG (directed acyclic graph) called the *condensation graph* or *component graph*.

6.2 DAGs and Topological Ordering

Definition

A **DAG** (Directed Acyclic Graph) is a directed graph with no directed cycles. A **topological ordering** of G is a linear ordering $v_1, v_2, \ldots, v_n$ such that for every edge (v_i, v_j), we have $i < j$ (edges go "forward" in the ordering).

Statement 3.19

In every DAG, there is at least one node with no incoming edges (a source, in-degree 0).

Proof by Contradiction

Suppose every node has at least one incoming edge. Start at any node and follow edges backward indefinitely. Since there are finitely many nodes, some node repeats, forming a directed cycle — contradicting the DAG property.

Statement 3.20

If G is a DAG, then G has a topological ordering.

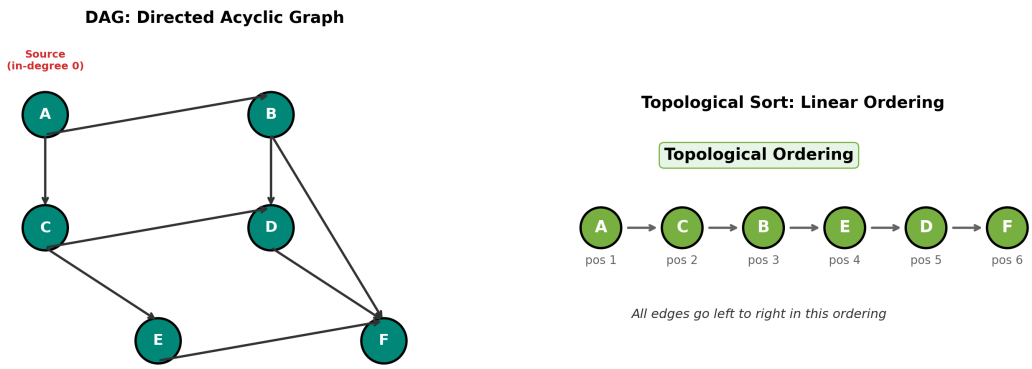

Figure 8: *Every DAG has at least one source (left). Topological ordering places all sources first, with all edges pointing forward (right).*

Proof by induction (using 3.19): Base $n = 1$ is trivial. For $n = k + 1$: by (3.19), find a source v, remove it to get G' (still a DAG). By induction, G' has a topological ordering. Place v first — since v has no incoming edges, this is valid.

Kahn's Algorithm for Topological Sort

```
Compute in-degree of all nodes
Initialize queue with all sources (in-degree 0)
While queue not empty:
    u = dequeue()
    Output u
    For each neighbor v of u:
        Decrement in-degree of v
        If in-degree[v] == 0: enqueue(v)
If all nodes output: valid topological order
Else: graph has a cycle
```

Relationship: (3.19) is the "engine" — the existence of a source lets us build an ordering. (3.20) follows directly by repeatedly applying (3.19). DAG $\Leftrightarrow$ topological ordering exists.

6.3 Viva Questions and Answers

Viva Q&A — SCCs and DAGs

Q: Is the topological ordering unique?

A: No. A DAG can have many topological orderings. Example: two disconnected nodes — either order works.

Q: Does topological sort exist iff the graph is a DAG?

A: Yes. DAG $\Rightarrow$ topological sort (proven above); topological sort $\Rightarrow$ no cycles (a cycle would require a node to appear before itself).

Q: Time complexity of topological sort?

A:$O(m+n)$ using Kahn's algorithm or DFS-based approach.

Q: What if the graph has exactly one cycle?

A: No topological ordering exists. Kahn's algorithm will stall with nodes remaining but no source available.

Q: Can a single node be an SCC?

A: Yes, if it has no self-loop and no cycles through it. Every node is in at least its trivial SCC.

Q: Do SCCs partition the edges too?

A: No. Only vertices are partitioned. Edges between different SCCs form a DAG (the condensation graph).

7 Greedy Algorithm Design

7.1 Core Elements and Proof Strategy

A greedy algorithm makes the locally optimal choice at each step with the hope of finding a global optimum. Two key ingredients must be proven:

Key ingredients for greedy algorithms

Ingredient	What it means	Why needed
Greedy-choice property	A globally optimal solution can be built from locally optimal (greedy) choices	Without this, local greed fails globally
Optimal substructure	Optimal solution contains optimal solutions to subproblems	Ensures we can solve optimally after each greedy choice

Standard Proof Technique: The Exchange Argument

1. Assume an optimal solution `OPT` to the problem
2. Show that if `OPT` doesn't include the greedy choice, you can modify `OPT` by "exchanging" some element for the greedy choice
3. Prove the modified solution is still feasible and optimal
4. Conclude: there exists an optimal solution containing the greedy choice

Example (Activity Selection): Let a_m be the activity with earliest finish time. Let `OPT` be any optimal solution. If $a_m \in$ `OPT`, done. If not, replace the first activity in `OPT` with a_m — the solution remains feasible (compatible) and same size, hence optimal.

Efficiency: Greedy algorithms are often efficient due to: (1) Preprocessing (sorting) — $O(n \log n)$, (2) Priority queues for fast greedy selection, (3) Single pass after preprocessing.

7.2 Greedy vs Dynamic Programming

Greedy vs Dynamic Programming

Aspect	Greedy	Dynamic Programming
When it works	Greedy-choice property holds	Optimal substructure + overlapping subproblems
Exploration	One choice, commit immediately	Multiple choices, evaluate all
Subproblems	Only one subproblem remains after greedy choice	Typically multiple subproblems per choice
Speed	Faster but may fail to find optimum	Slower but always finds optimum
Memoization	Not needed	Often needed for overlapping subproblems
Examples	Activity selection, Huffman coding, fractional knapsack, Dijkstra, MST	0/1 Knapsack, LCS, edit distance, matrix chain multiplication

Classic Failure Case: 0/1 Knapsack. The greedy-choice property fails — taking the highest value-per-weight item first doesn't always yield the optimal solution. DP is required.

7.3 Viva Questions and Answers

Viva Q&A — Greedy Algorithms

Q: How do you prove a greedy algorithm is correct?

A: Prove the greedy-choice property (there exists an optimal solution containing the greedy choice) and optimal substructure (after the greedy choice, the remaining problem has optimal substructure). Use the exchange argument.

Q: When does the greedy-choice property fail?

A: When locally optimal choices don't combine into a global optimum. Example: 0/1 Knapsack vs fractional knapsack.

Q: Is greedy always faster than DP?

A: Generally yes, but the key difference is guarantee. Greedy is faster but may fail; DP is slower but always optimal when applicable.

Q: Can every DP problem be solved with greedy?

A: No. Only when the greedy-choice property additionally holds. DP problems like 0/1 knapsack require exploring all choices.

8 Shortest Path Algorithms

8.1 Dijkstra's Algorithm

Statement 4.15

Using a priority queue, Dijkstra's algorithm runs in $O(m)$ time plus the time for n ExtractMin and m ChangeKey operations.

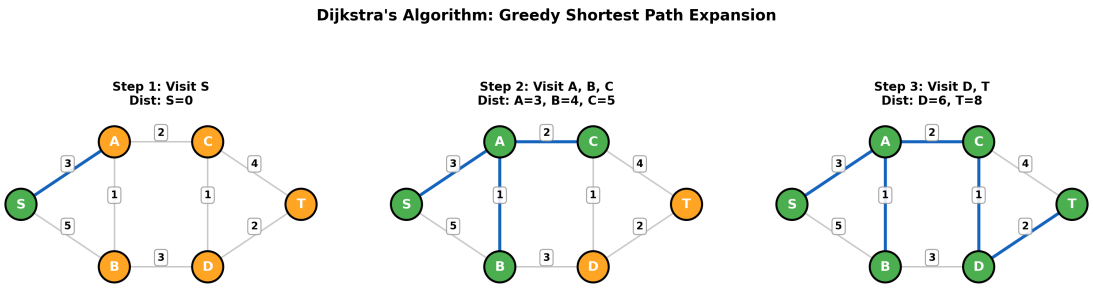

Figure 9: *Dijkstra's algorithm greedily expands the frontier: at each step, the node with minimum tentative distance is finalized.*

Dijkstra's algorithm maintains a set of "finalized" nodes whose shortest-path distances from the source are known. At each step, it extracts the node with minimum tentative distance and relaxes all outgoing edges.

Priority queue implementations for Dijkstra

Data Structure	ExtractMin	ChangeKey	Total Time
Array	$O(n)$	$O(1)$	$O(n^2)$
Binary Heap	$O(\log n)$	$O(\log n)$	$O(m \log n)$ or $O((m+n)\log n)$
Fibonacci Heap	$O(\log n)$ amortized	$O(1)$ amortized	$O(m + n \log n)$ — best known

8.2 When Dijkstra Fails

Conditions for Dijkstra's failure

Condition	Why it fails	Solution
Negative edge weights	Greedy assumption breaks: a shorter path might be found later through a longer current path that has negative edges ahead	Use Bellman-Ford ($O(nm)$) or SPFA
Negative cycles	Path length can decrease indefinitely; shortest path is undefined	Bellman-Ford detects and reports

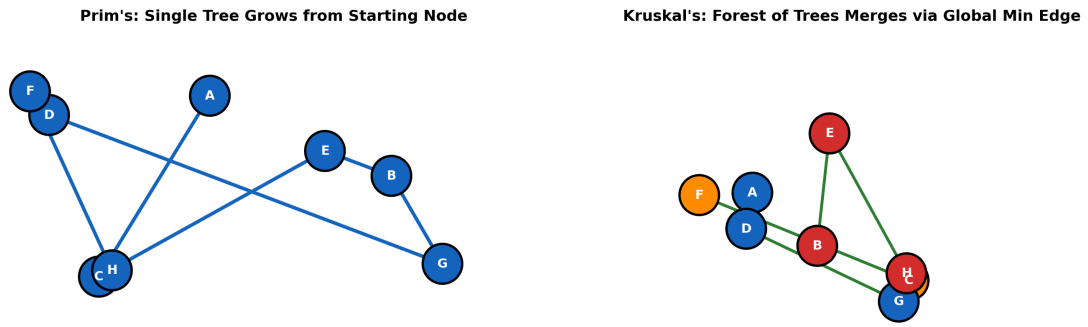

Figure 10: *Left: Prim's grows a single tree from a starting node. Right: Kruskal's maintains a forest that merges via globally minimum edges.*

8.3 Viva Questions and Answers

Viva Q&A — Shortest Paths

Q: Why does Dijkstra fail with negative weights?

A: Dijkstra greedily finalizes nodes, assuming no shorter path will be found later. With negative weights, a longer current path plus a negative edge ahead can beat a previously finalized distance.

Q: Can Dijkstra work with negative weights if no negative cycles?

A: No. The greedy finalization is fundamentally broken by any negative edge, even without negative cycles.

Q: What is the best-known time complexity for Dijkstra?

A:$O(m + n \log n)$ with Fibonacci heaps, which is optimal for comparison-based priority queues.

Q: How does Dijkstra differ from Prim's algorithm?

A: Dijkstra picks edges by minimum distance from the source ($d[u] + w(u, v)$). Prim picks edges by minimum weight connecting the tree to outside nodes. Both use greedy approach with priority queues but optimize different objectives.

9 Minimum Spanning Trees

9.1 Cut Property and Cycle Property

These are the two fundamental theorems governing MSTs. Every correct MST algorithm (Prim's, Kruskal's) implicitly uses one of these properties.

Cut Property

Let $(S, V \setminus S)$ be any cut of G. Let e be a minimum-weight edge crossing the cut. Then e belongs to some MST. If the minimum is unique, e belongs to every MST.

Cycle Property

Let C be any cycle in G. Let e be a maximum-weight edge in C. Then e does not belong to any MST. If the maximum is unique, e is excluded from every MST.

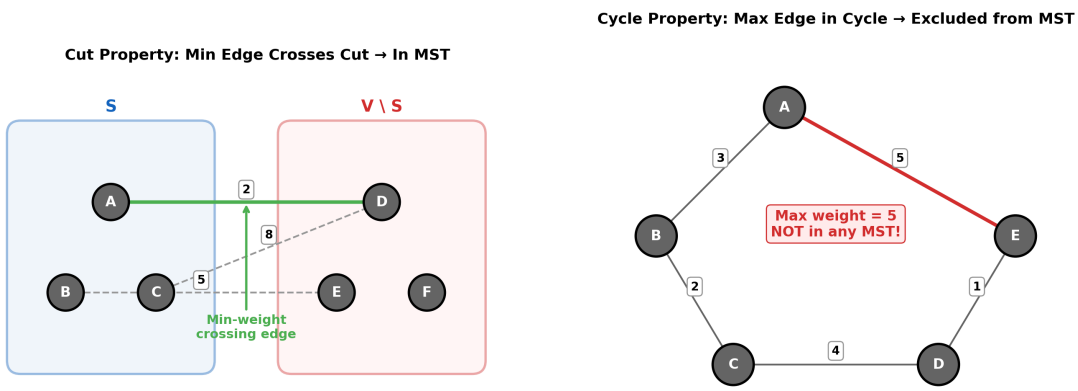

Figure 11: *Left: Cut Property — the minimum-weight edge crossing any cut must be in the MST. Right: Cycle Property — the maximum-weight edge in any cycle is never in the MST.*

Proof of Cut Property (Exchange Argument)

Suppose MST T does not contain minimum-weight crossing edge $e = (u, v)$. Since T is spanning, it contains some edge e' crossing the same cut. Replace e' with e: $T' = T \cup \{e\} \setminus \{e'\}$ is still connected and spanning. Since $w(e) \leq w(e')$, we have $w(T') \leq w(T)$. Since T was minimum, T' is also an MST containing e.

Proof of Cycle Property

Suppose maximum-weight cycle edge e is in some MST T. Remove e from T — this disconnects T into two components. The cycle C must contain another edge e' reconnecting these components. Since e is maximum-weight, $w(e') \leq w(e)$. Replacing e with e' gives a spanning tree with weight $\leq w(T) - w(e) + w(e') \leq w(T)$. If $w(e') < w(e)$, this contradicts T being minimum.

9.2 Prim's vs Kruskal's Algorithm

Comparison of Prim's and Kruskal's algorithms

Feature	Prim's Algorithm	Kruskal's Algorithm
Approach	Single tree grows from start node	Forest of trees merges via global min edge
Greedy criterion	Min edge from tree to non-tree node	Min edge that doesn't form a cycle
Data structure	Priority queue (distances/weights)	Union-Find (disjoint sets)
Best for	Dense graphs ($m \approx n^2$)	Sparse graphs ($m \ll n^2$)
Time (basic)	$O(n^2)$ with array	$O(m \log m)$ with sorting
Time (optimized)	$O(m \log n)$ with binary heap; $O(m + n \log n)$ with Fibonacci	$O(m\,\alpha(n))$ with Union-Find path compression
Negative weights?	Works fine	Works fine
Disconnected input?	Finds MST of one component	Finds Minimum Spanning Forest

Why Prim's and Kruskal's don't "fail" like Dijkstra: MST works fine with negative weights. The greedy choice (minimum edge) is safe because of the Cut Property, which holds regardless of the sign of weights. There is no "distance accumulation" issue like Dijkstra's path lengths.

9.3 Viva Questions and Answers

Viva Q&A — Minimum Spanning Trees

Q: When can we guarantee an edge is NOT in any MST?

A: When it is the unique maximum-weight edge in some cycle (Cycle Property). More generally, if it is the heaviest edge in any cycle, it cannot belong to any MST.

Q: When to use Kruskal's vs Prim's?

A: Kruskal's for sparse graphs (edge list given, $m \ll n^2$). Prim's for dense graphs (adjacency matrix given, $m \approx n^2$). Kruskal's requires sorting edges; Prim's doesn't.

Q: Does MST work for directed graphs?

A: No. MST is for undirected graphs. For directed graphs, the problem is Minimum Spanning Arborescence (Edmonds' algorithm).

Q: If there are multiple edges with the same maximum weight in a cycle, can any be in an MST?

A: Yes. The guarantee only holds for the *unique* maximum. If multiple edges tie for maximum, some (but not all simultaneously) may appear in an MST.

Q: Why does Kruskal's skip edges that form cycles?

A: That edge would be the heaviest in the cycle formed (since all lighter edges were already selected), so by the Cycle Property, it's not in any MST.

10 Algorithm Comparison Summary

Summary of all algorithms covered

Algorithm	Problem	Approach	Time	Key Limitation
BFS	Shortest path (unweighted), connectivity, bipartite test	Queue-based level-order	$O(m+n)$	Unweighted graphs only
DFS	Cycle detection, topological sort, SCCs	Stack/recursion, deep dive	$O(m+n)$	No shortest path guarantee
Dijkstra	Single-source shortest path	Greedy with priority queue	$O(m + n \log n)$ (Fibonacci)	Negative weights break it
Bellman-Ford	Single-source shortest path	Dynamic programming relaxation	$O(nm)$	Slower but handles negatives
Prim's	Minimum Spanning Tree	Grow single tree, min crossing edge	$O(m + n \log n)$ (Fibonacci)	Undirected graphs only
Kruskal's	Minimum Spanning Tree	Sort edges, Union-Find	$O(m\,\alpha(n))$ after sort	Undirected graphs only
Kosaraju's	Strongly Connected Components	Two DFS passes	$O(m+n)$	Directed graphs only
Kahn's	Topological sorting	Repeatedly remove sources	$O(m+n)$	Graph must be a DAG

Core Insight: Greedy algorithms (Dijkstra, Prim's, Kruskal's) work when local choices are globally optimal. When the greedy-choice property fails (0/1 Knapsack, negative weights in shortest paths), Dynamic Programming or other approaches are needed.

Key Differences: Dijkstra vs Prim vs Kruskal

- ***Dijkstra:*** *Minimizes path distance from source ($d[u] + w(u, v)$). Grows set of nodes with finalized shortest paths.*
- ***Prim's:*** *Minimizes edge weight connecting tree to outside. Grows a single connected tree.*
- ***Kruskal's:*** *Minimizes global edge weight without forming cycles. Maintains a forest that eventually connects.*

Viva Q&A — Final Review

Q: What do you mean by optimality in algorithms?

A: Optimality means the solution achieves the best possible value according to the objective function: minimum cost, maximum profit, shortest path, etc. A proof requires showing no other feasible solution has a better value.

Q: Why use DP rather than greedy?

A: Use DP when the greedy-choice property fails — locally optimal choices don't combine into a global optimum. DP explores all choices and combines results; greedy commits to one choice immediately.

Q: When does Prim's "fail"?

A: Prim's doesn't truly "fail" — it always produces an MST for a connected graph. If the graph is disconnected, it finds an MST for the component containing the start node. Kruskal's handles disconnected graphs by producing a Minimum Spanning Forest.

Q: Can Kruskal's stop early?

A: Kruskal's must process edges until $n - 1$ edges are selected (or until the forest connects). Prim's can stop once $n - 1$ edges are selected since it grows a single tree.